```
  c ←
s[0]
         #2
  if node.children[c] == null then
                           #3
    return (null, “”, s,
null)                                                  #4
  else
    edge ←
node.children[c]
    #5
    prefix, suffixS, suffixEdge ← longestCommonPrefix(s,
edge.label)          #6
    return (edge, prefix, suffixS,
suffixEdge)                                  #7
```

#1 Method `matchEdge` takes an `RTNode` and the string key `s` to be matched; it returns a tuple with the edge matched, if any, the common prefix between `s` and the edge’s label, and the suffixes of those strings. We assume that `node` is never `null` and that `s` is not empty.

#2 Since `s` is not the empty string, it will certainly have a first character `c`.

#3 Look up in the hash table for children if there is any edge whose label starts with `c`.

#4 If there isn’t any, it means there is no edge with a common prefix to `s`. Then, it returns `null` for the edge, an empty string for the common prefix, and consequently compute the suffixes.

#5 Retrieves the outgoing edge in `node` starting with character `c`.

#6 Computes the longest common prefix between `s` and edge’s label, and the remaining suffixes.